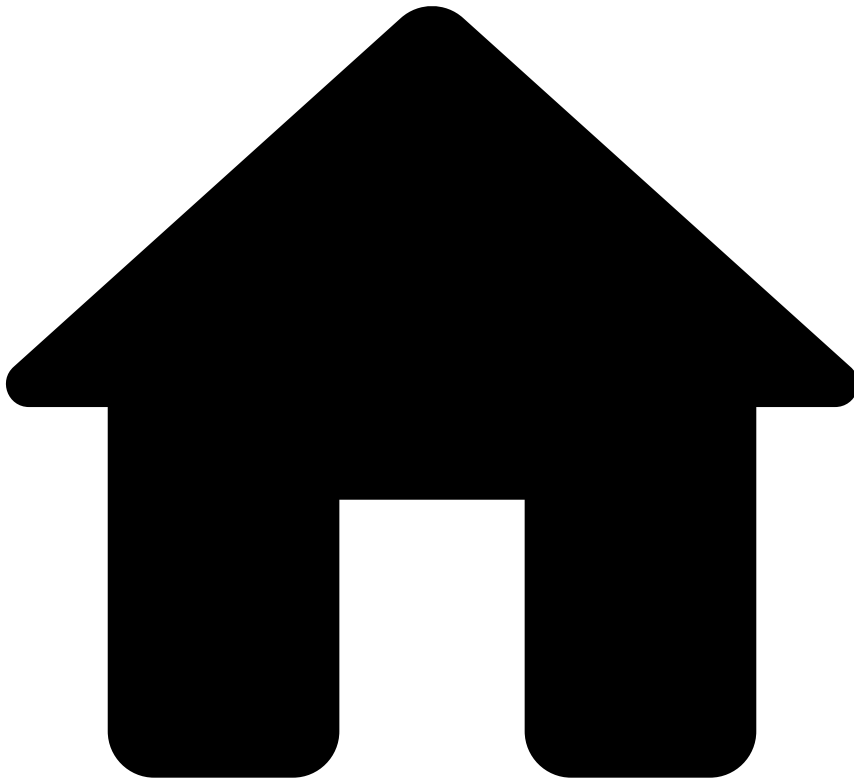


-



- Securing an Application
- Security Entities

On this page

Was this page helpful? 

Security Entities

This section gives you a high-level overview of the main security components or entities that exist in Pulse.

Security in Pulse has four core entities, each with its purpose:

- Permissions
- Roles
- Groups
- Users

Together they provide a flexible and comprehensive security model.

Permissions🔖🔖

In Pulse, permissions are used for controlling access to resources. For example, the permission `application:apps:create` allows the creation of a new application.

Pulse already comes with a list of predefined internal permissions. They can be used when creating roles.

A full list with all the permissions in Pulse can be found on the [List of Permissions](#) page.

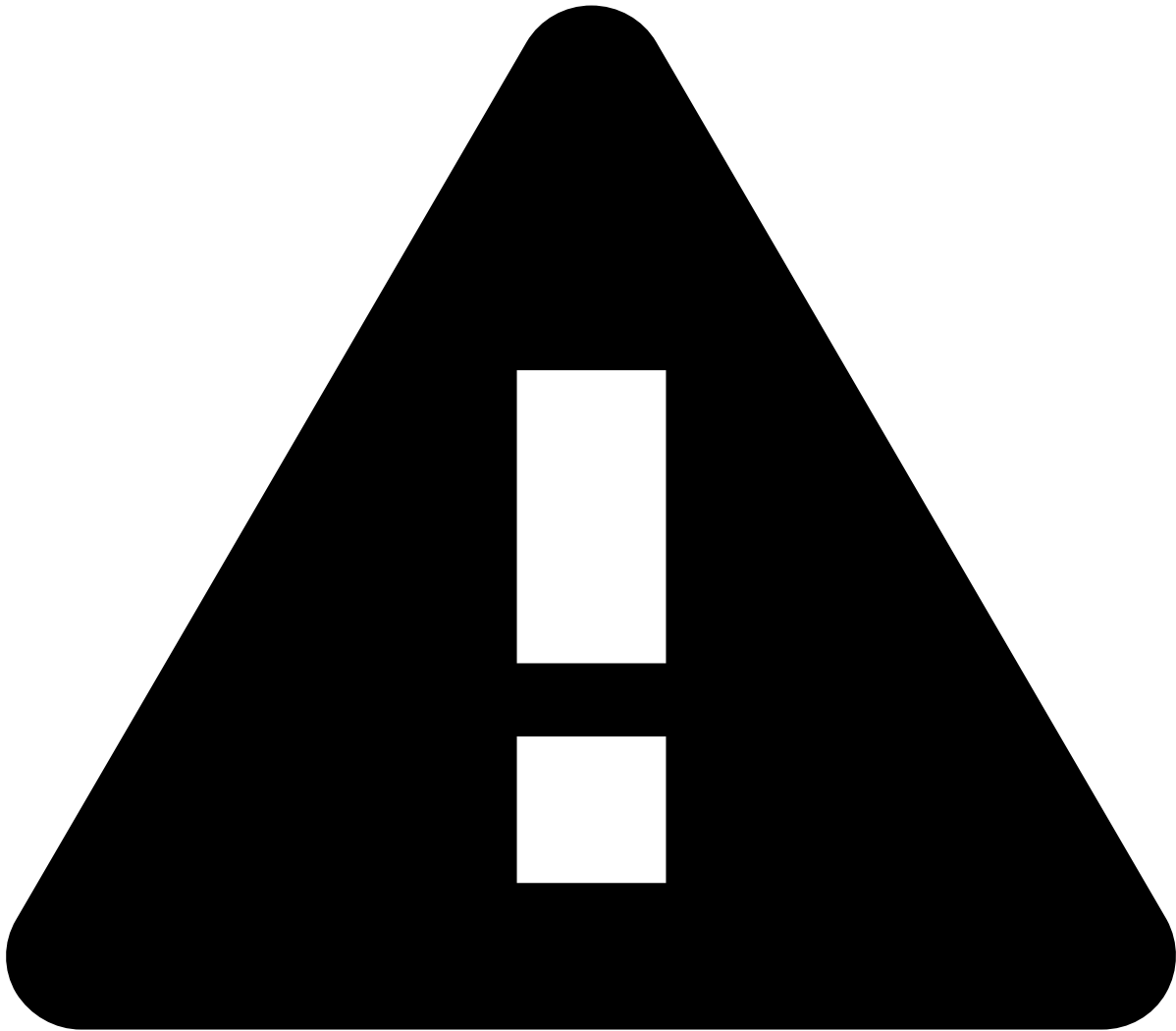
Permission contexts🔖🔖

Permissions have the following structure: `<context>:[<resource>]:[<right>]`.

For example, to allow a user to create a sandbox evaluations within the application context, the user must have the `application:sandbox_evaluations:create` permission.

The permission contexts are the following:

- **admin** - All things related to administration (e.g. configuration, deployment, etc). This context is intended only for system administrators.
- **application** - This is the context of all resources (e.g. data science models, rules, schemas, etc).
- **pulseviews** - This is the context for UI-related actions (e.g. login, tab management, etc).
- **security** - Security-related actions belong to this context (e.g. creating users, groups, etc).



Warning

To log in, users must have the `pulseviews:login` permission.

Roles🔗🔗

A role is a set of permissions. Each user or group can have many roles. If a user belongs to a group, it has all the roles of that group.

Roles are typically created to ease the management of permissions. For instance, the update permissions of different resources can belong to a single role called *updater-role*. From now on, we can use the *updater-role* to refer to all update permissions, instead of using individual permissions.

Default roles🔗🔗

Pulse already has some roles created by default. These roles can be selected when creating a new user from the Administration view:

- **admin** - The administration role. It has permissions to do everything.
- **pulseviews** - This role can view, update and delete all resources, but it only has view rights over security.
- **pulseviews-reader** - This role can only read resources, nothing more.
- **developer** - The user with this role has permissions to do everything except system administration.

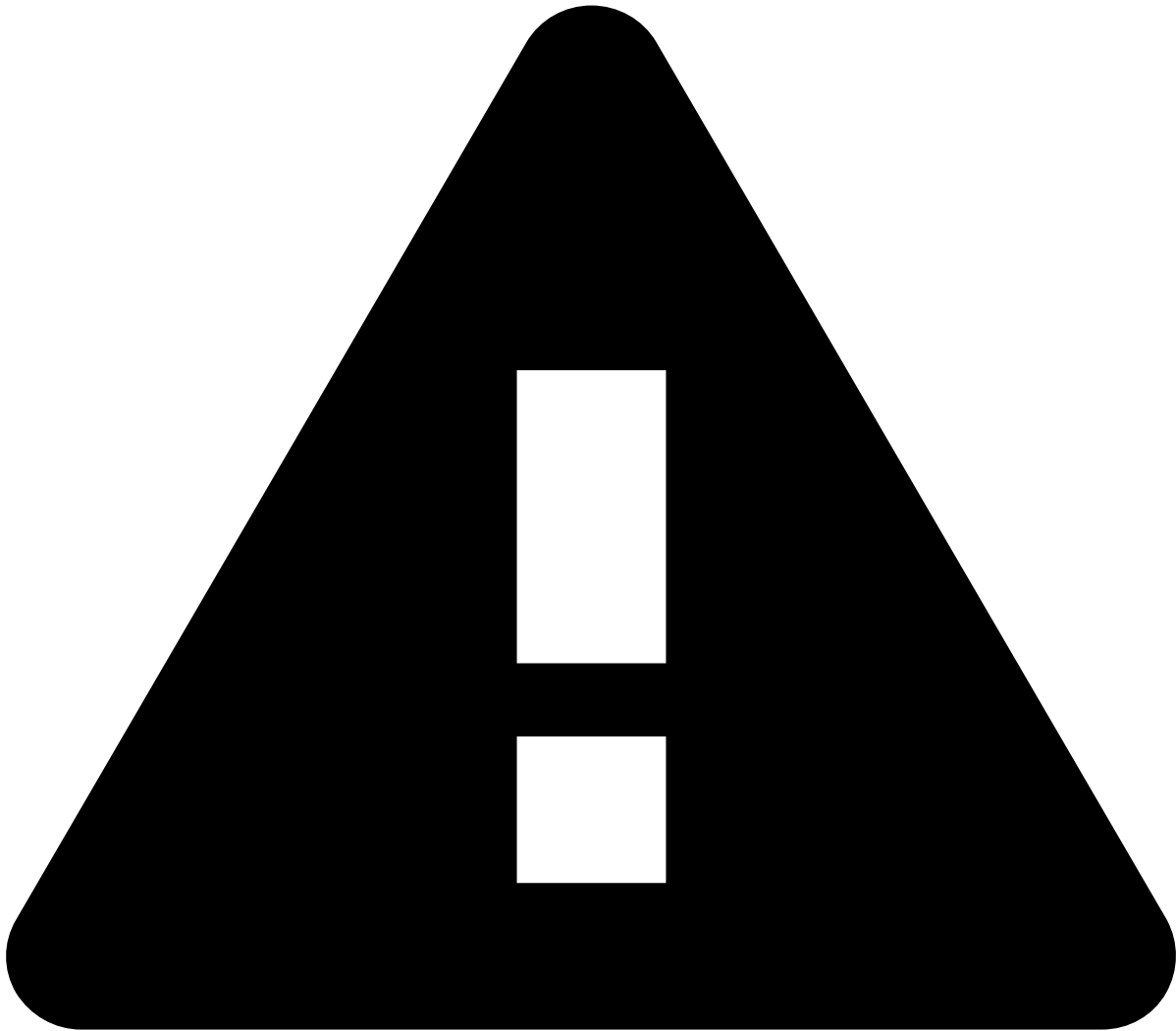
The following table shows the permissions for each default role.

Permission name	Permission	pulseviews	pulseviews-reader	developer	admin
Group hierarchy read	security:ownership:read	(tick)	(tick)		
Role read	security:roles:read	(tick)	(tick)		
Permissions read	security:permissions:read	(tick)	(tick)		
Tenant read	security:tenants:read	(tick)	(tick)		
Tenant domain read	security:tenantdomains:read	(tick)	(tick)		
Pulseviews all	pulseviews:*	(tick)			
Pulseviews login	pulseviews:login		(tick)		
Application all CRUD	application:*	(tick)			
Application all read	application:*:read		(tick)		
Management tabs all	pulseviews:management:tabs:*		(tick)		
History read (single resource)	application:history:resource:read		(tick)		
History read (application-wide)	application:history:global:read		(tick)		
Everything	*			(tick)	(tick)

Groups

A group is a security component that can have multiple roles. Not only that, they also provide a hierarchical way to control access to resources.

The hierarchical structure of groups is what makes them a powerful authorization abstraction, because **a group can do, at least, everything that its descendants do.**



Roles are not propagated

If User A belongs to a child group that can update a given hierarchy, User B, belonging to a parent group, will also be able to do it. The only exception to this rule is if User B does not have the static permission to update hierarchies. Groups entitle static permissions via roles but these are not propagated in any way through the group hierarchy.

Theoretical use case

An easy way to understand how groups behave is to map them to an hypothetical company organizational positions. Each group is a position occupied by some employees. An employee in a management position can see the work items of its subordinates, but the reverse is not true.

Consider the following positions of an hypothetical company.

And consider some of that company employees connected to their respective positions. The connections between users and groups can be read as: Lisa is a Managing Director.

Now imagining Lisa has some reports she wants to share with specific positions, lets see how she can accomplish that using groups.

- **Human Resources Report** - Peter, Kate and any future HR worker should be able to see it, so this report should be visible to the group '*HR Worker*'. Notice that both Carl and Lisa will be able to see this report. Also notice that besides being respectively '*HR Salaries Worker*' and '*Hiring Worker*', both Peter and Kate must also be '*HR Worker*'.

- **Salaries Report** - This report should be visible for all workers responsible by salaries, so it should be visible to the groups '*Finance Salaries Worker*' and '*HR Salaries Worker*'. This report will be visible to Arthur, Mary, Peter and their superiors, John, Carl and Lisa.
- **Arthur Performance Report** - This report should only be visible to Arthur and his bosses. This is not a good use-case for groups because in order to solve it, a new child group of '*Finance Manager*' called '*Arthur (Finance)*' has to be created and linked to Arthur. If the report is then made visible to this group, both Arthur and his superiors will be able to see it.

Example use case

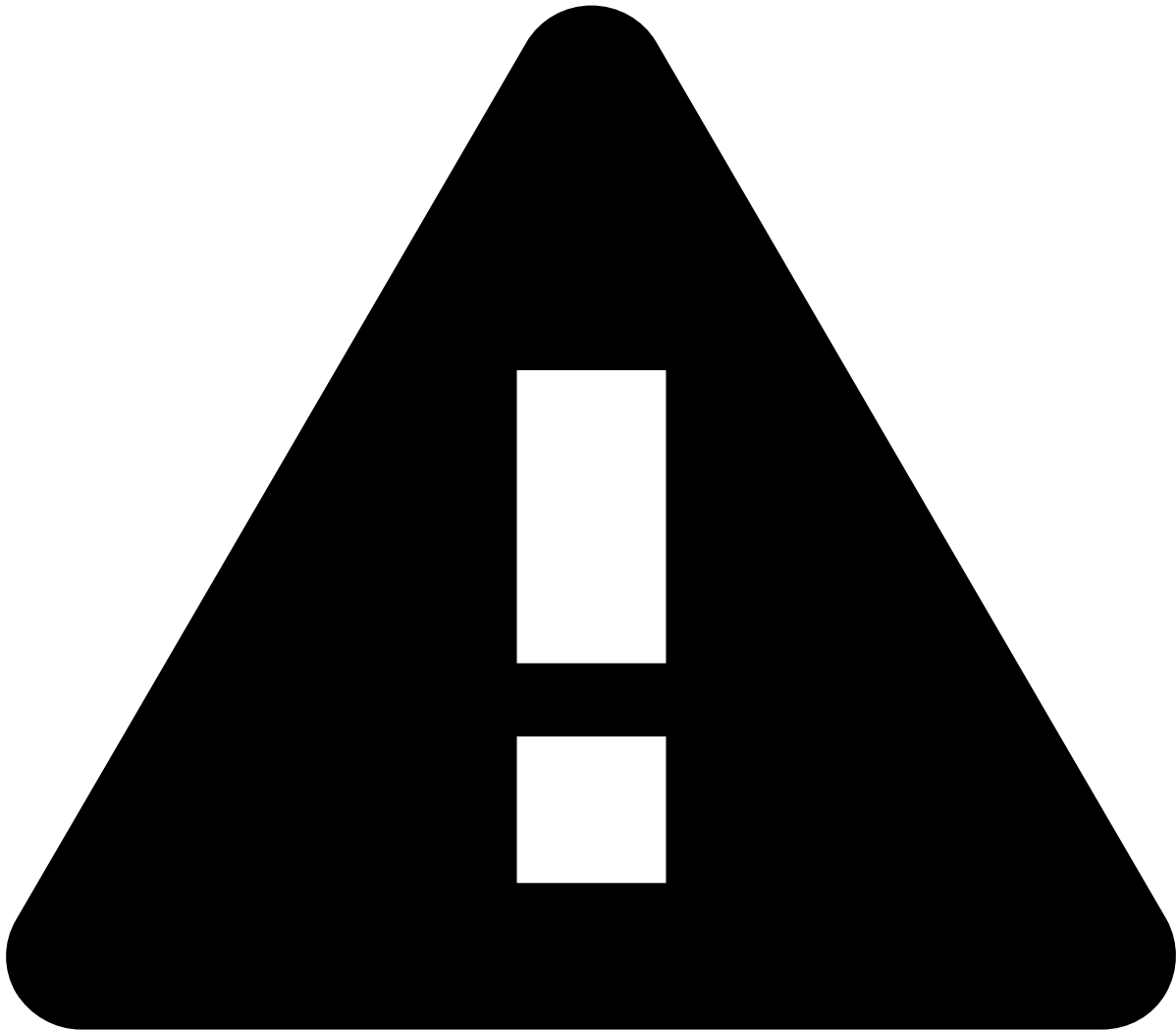
Consider the following scenario, a card brand company provides different data to issuers and acquirers, in their turn, each acquirer will only have access to a subset of that data, and so on. A possible groups hierarchy to map this scenario could be as follows.

Considering the structure above, deploying Pulse in this company to **provide services to other companies** is just a matter of assigning users to each position and sharing them with other companies. Each application resource must then be given the correct visibility so that only users of those particular groups can see it.

Users

A user is usually associated with "someone" using Pulse. It is used to prove the entity is who it says it is, through **authentication**, and to make sure it has the rights to do what it is trying to do, through **authorization**.

Users can have both **roles** and **groups**. Roles are usually associated with a user to define its **static permissions**. That is, permissions that **always** apply to a user, and that cannot be controlled for each application or resource instance. On the other hand, groups can be used to specify **per-instance permissions**, and are used to control the visibility a user will have over each resource instance, see [Securing Metadata](#).



warning

A user can be authenticated through external authentication. In this case, the password field is not required.

Managing users and credentials

Pulse has several features designed to help manage users and their credentials. These features need to be explicitly configured if you want to use them.

There are several features designed to prevent access by unauthorized users. These features block a user thus preventing him from signing in and use his account. The user is informed that his account is blocked when he tries to log in. Users can be blocked by:

- Explicitly blocking them in the user page in the administration section of the user interface.
- Users that do not login for a given period of time can be blocked automatically.
- Block the user after multiple failed login attempts.

After being blocked, users need to be enabled by an administrator in the web user interface administration section.

There are also several features designed to enforce that users have secure credentials:

- Password strength verification. Passwords are checked to contain three or more of the following types of characters:
 - 1 upper case letter

- 1 lower case letter
- a number (0-9)
- a special character (!@#\$\$%^&*)
- Password uniqueness verification. It is not possible for a user to re-use one of their previous passwords.
- Administrators can mark the password of a user as expired. This will prompt the user to insert a new password before he can log in. Alternatively, the user password can also be changed in the administration user page.
- Password age verification. If the user did not change the password in a given period of time, the password is automatically marked as expired.

Other features include:

- Invalidate user session after a period of inactivity.
- Log authentication related events to the audit log:
 - login
 - logout
 - session expiration
 - account disable
 - password expire
 - password change
 - user creation

Default users

Users in Pulse can be disabled due to a variety of reasons, including too many failed login attempts, password expiration or inactivity. However, Pulse relies on making internal calls for communicating between the Pulse Servers and its App Engines to function properly, and those calls require authentication.

In order to handle internal and external authentication and ensure PCI-DSS compliance, Pulse creates the following two users on startup:

- **pułse** : User with full permissions for external authentication, e.g. in Pulse APIs.
- **system** : User with full permissions that cannot be used for external authentication, i.e. it is impossible to log in to Pulse. This user is exclusive to internal Pulse usage, so you should not try to use it.